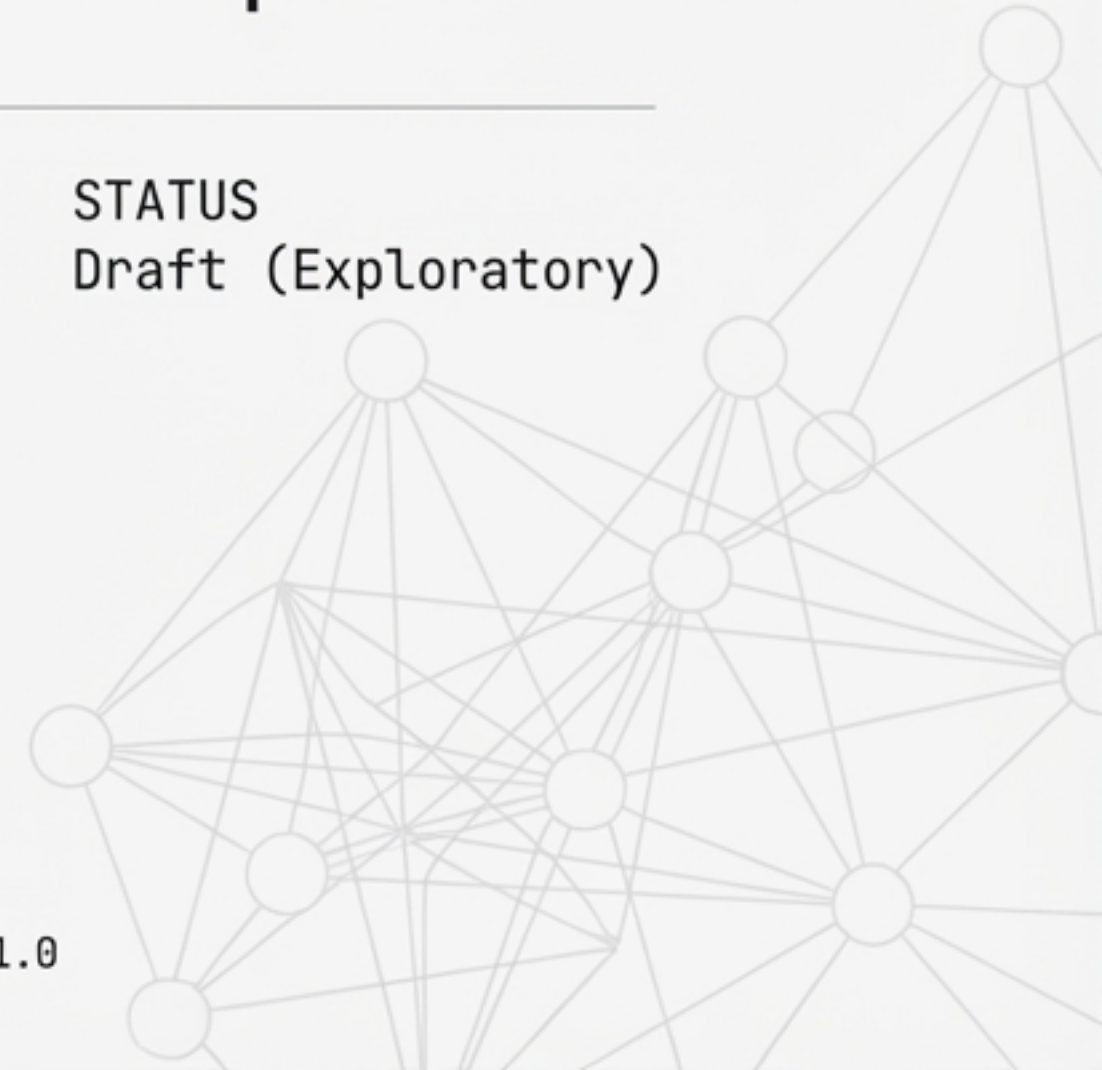


Semantic Testing DSL

Unit Tests for Content & Knowledge Graphs

DOCUMENT	VERSION	DATE	STATUS
issues-fs__semantic-testing-dsl	v1.0	2026-02-05	Draft (Exploratory)

DEPENDENCIES: issues-fs__semantic-graph-code-representation v1.0, issues-fs__semantic-text-architecture v1.0



The Gap Between Code Rigour and Content Ambiguity



The Problem:

1. Current tools test the paint (text representation), not the structure.
2. Valid editorial polishing (e.g., 'remediation' -> 'mitigation') triggers false failures.
3. Result: High maintenance costs and test fatigue.

Testing the Skeleton, Not the Skin

X-Ray

The project risk must be addressed by a remediation plan. The legal document is or to rework better project risk m remediation p themirement.



The state the lug revs ar requirement adapbation of the hund, develop untaraatters to provev that the project release the are proper solution, problem of as identified and

Change Type	Example	Graph Impact	Test Result
Text Change	"remediation" → "mitigation"	None (Same Node Type)	Pass
Text Change	"must" → "should"	Property "modal" changed	Fail (Strictness Check)
Text Change	Deleted Citation	Edge "has_evidence" removed	Fail (Completeness Check)

Why a Purpose-Built DSL?

Finding the Goldilocks Zone between raw code and ambiguity.

Raw Python/JS

```
33 def resource_exists(self, resource):
34     self.test_session = TestSession('test_session')
35
36 def test_resource(self, resource):
37     get_test = self.test_session.get('test_session')
38     if resource_exists(resource):
39         self.test_session.add(resource)
40         break
41     for resource in resources:
42         self.test_session.add(resource)
43         self.test_session.commit()
44
45 def test_resource_exists(self, resource):
46     test_session = TestSession('test_session')
47     get_test = test_session.get('test_session')
48     if resource_exists(resource):
49         test_session.add(resource)
50         test_session.commit()
51     else:
52         test_session.add(resource)
53         test_session.commit()
```

Too Complex.

Intent buried in boilerplate.
Inaccessible to content
architects.

Semantic DSL

assert Risk exists

Just Right.

Optimized for
semantic assertions.
Compiles to typed code.

Gherkin

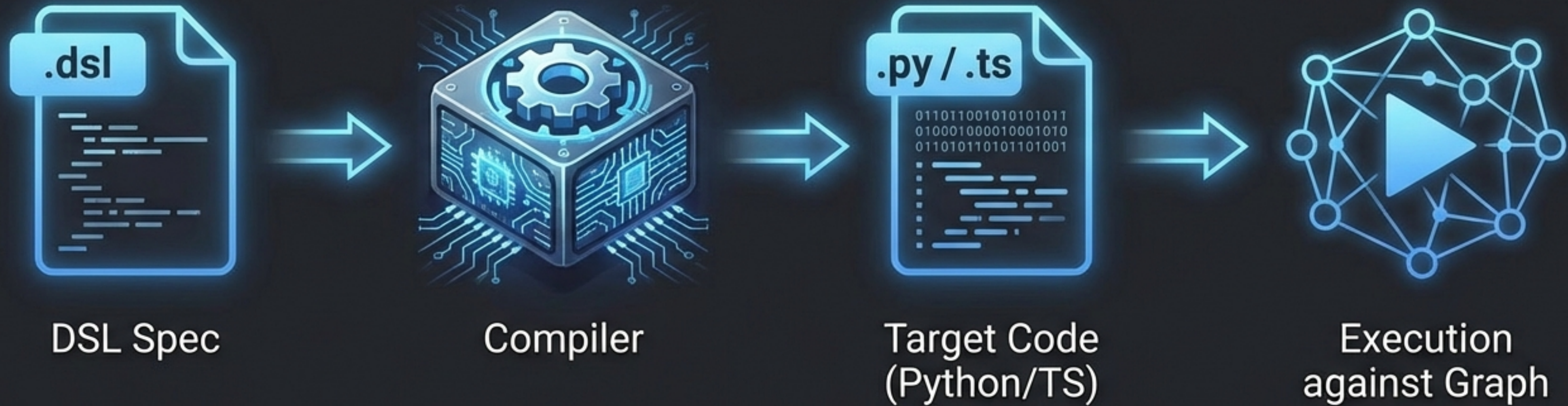
Given I see text...

Too Abstract.

English
pretending to be code.
Hides implementation
details.

The Compilation Pipeline

This is not a text parser. It is a code generator.



The DSL abstracts the complexity of graph traversal into readable rules.

Syntax Snapshot

Graph
Iteration

```
spec RiskManagement  
  
test 'Risks must have mitigations' {  
  for each Risk {  
    assert exists(in_edges(Mitigation))  
    or assert exists(out_edges(Mitigation))  
  }  
}  
  
test 'High Severity requires immediate action' {  
  for each Risk where severity == 'High' {  
    assert exists(Remediation)  
  }  
}
```

Scope
Declaration

Named Test
Block

Structural
Assertion

Abstraction Without Magic

The Input (DSL)

`assert Risk exists`

Clear Intent.

The DSL handles the heavy lifting of graph traversal.

The Output (Generated Python)

```
for node in graph.nodes:
    if isinstance(node, Risk):
        if not node.is_active:
            continue
        found = True
if not found:
    raise AssertionError('Risk node missing')
```

Robust Implementation.

This Is Not Gherkin

Testing structure, not user interface behaviour.

Gherkin (BDD)

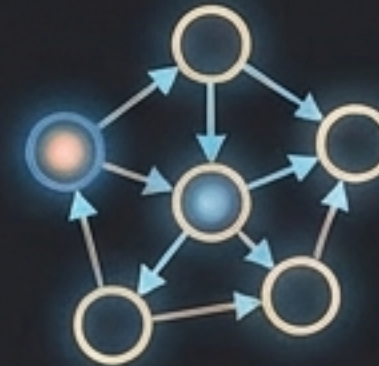


Focus: Tests UI Behaviour

Mechanism: Brittle Regex Glue

Example: "When I click 'Submit'..."

Semantic DSL



Focus: Tests Graph Structure

Mechanism: Deterministic Compilation

Example: "assert Risk has Remediation"

Brittleness:	High (Text Match)	Low (Structure Match)
Transparency:	Hidden Step Defs	Visible Compilation

Core Pattern: Structure & Relationships

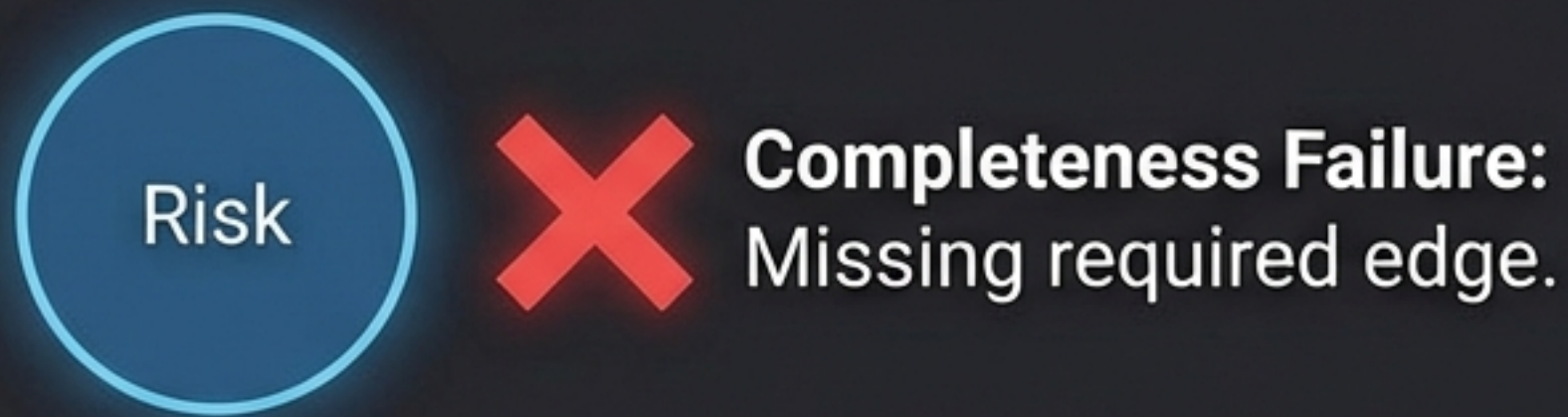
Relationship Validity



Pattern 1: Structural Completeness.
Verifying required elements exist.

Pattern 2: Relationship Validity.
Verifying nodes are connected correctly, regardless of the phrasing used in the text (e.g., "mitigated by", "addressed by").

Completeness Failure



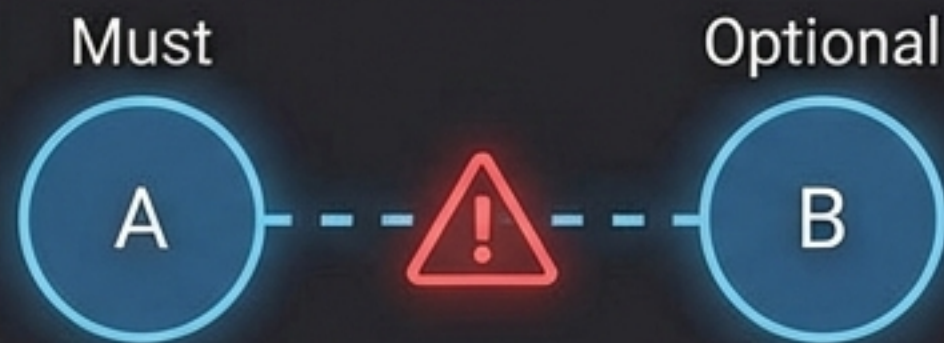
Advanced Patterns: Evidence & Consistency

Evidence Chains



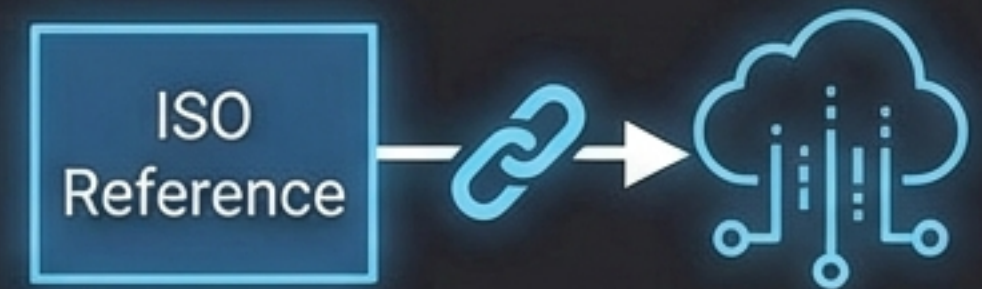
Verifying claims are supported by data.

Consistency Checks



Preventing logical contradictions (e.g., Must vs. Optional).

External Validation



Resolving and verifying external standard references.

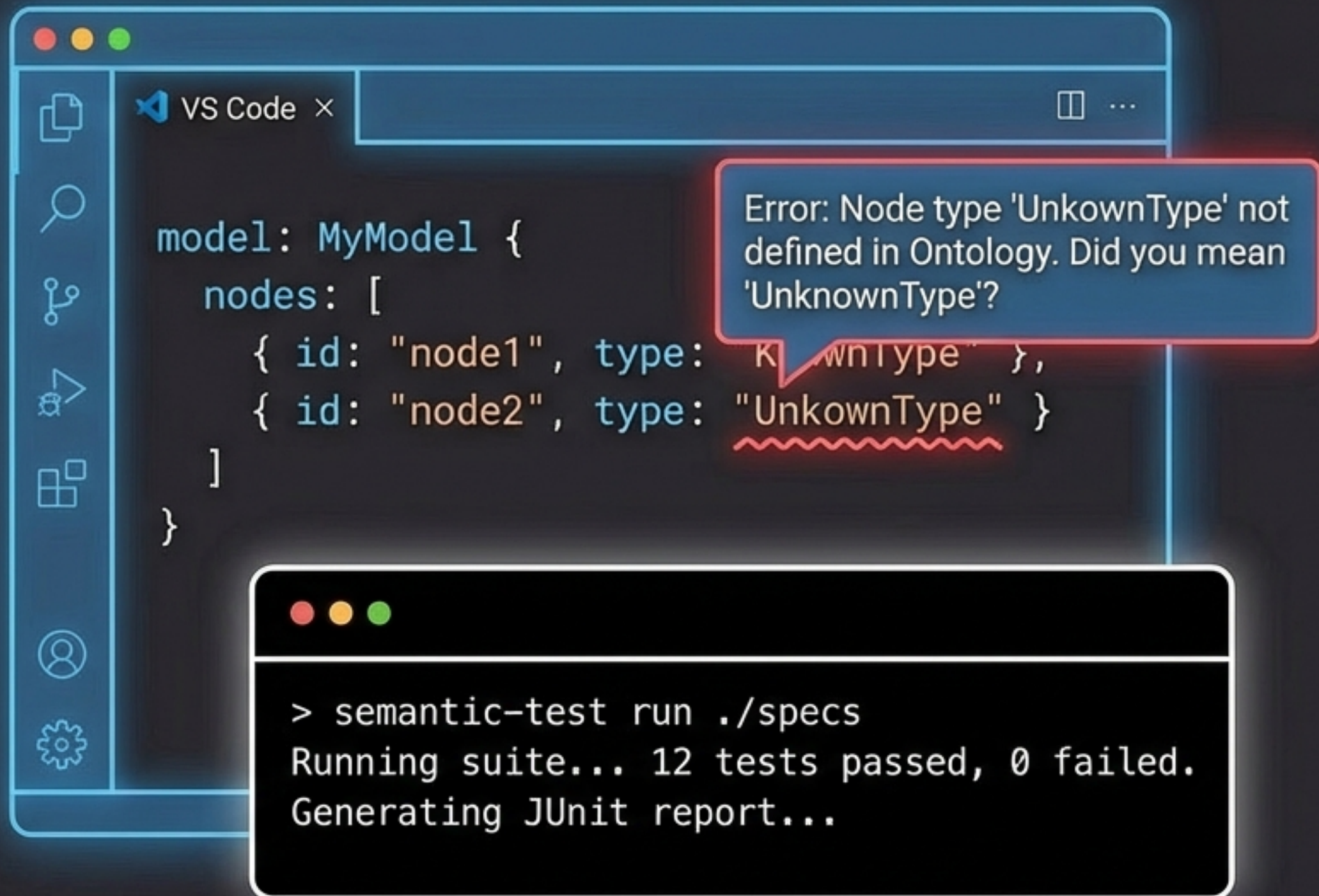
The Content TDD Lifecycle



The specification acts as a living guardrail.

Authors receive immediate feedback when content drifts from architectural requirements.

Developer Experience & CI/CD



- CLI Integration for GitHub Actions

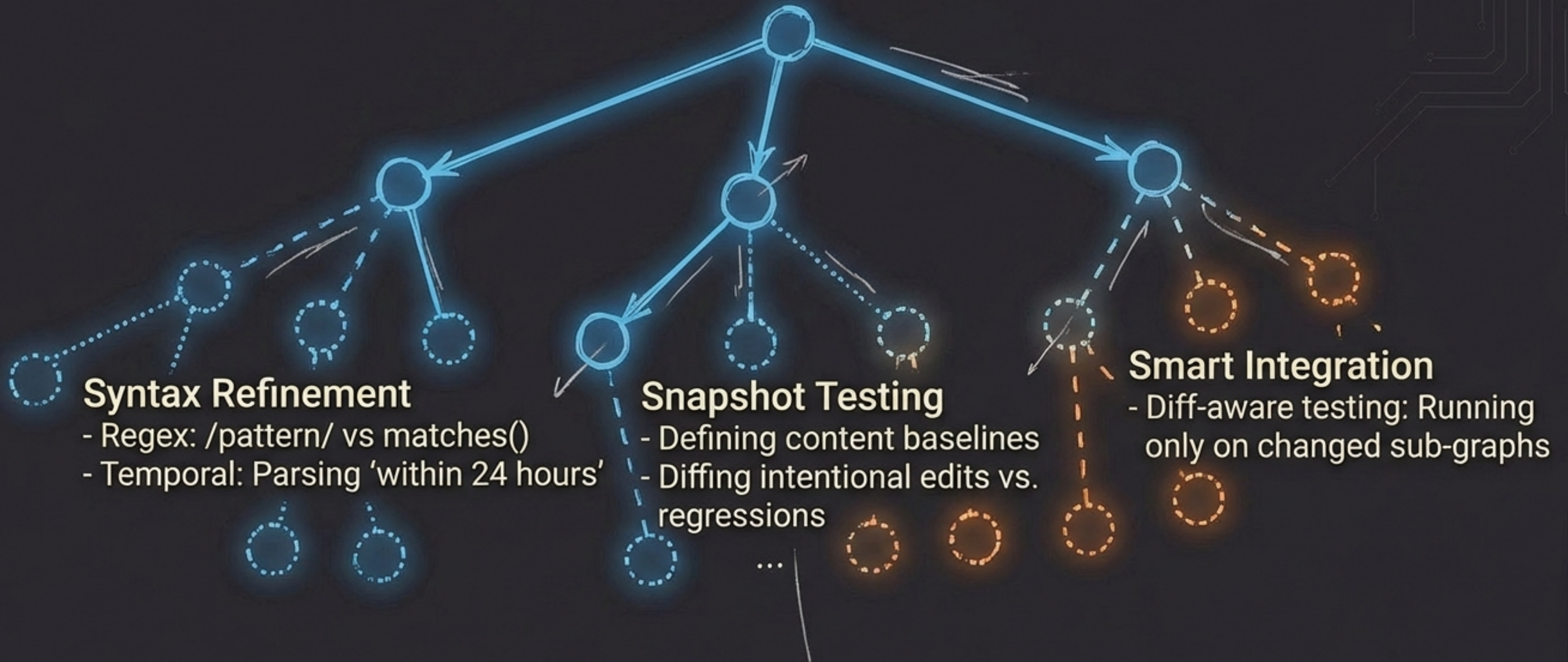


- IDE Support with Autocomplete



- Localized, helpful error messages

Areas for Exploration



Implementation Roadmap



From 'English as Code' to 'Code for Meaning'

Summary:

We are abandoning the brittleness of text matching for the robustness of structural testing. This applies software engineering discipline to content ambiguity.

Call to Action:

Review syntax proposals (Decisions TD5/TD6) and contribute to the Decisions Log.

